

Unit 4.3: Monte Carlo Integration and Importance Sampling

Dr. Manoj C. Patil

2026-09-30

Why Monte Carlo?

Quadrature (Unit 4.2) puts points on a grid. In d dimensions the grid needs (points per axis) ^{d} evaluations, which quickly becomes impossible.

Monte Carlo uses **random points** instead. Its error is about $1/\sqrt{n}$ **no matter how many dimensions** the integral has.

Monte Carlo Integration

The idea

An integral is an expectation, and an expectation is estimated by an average. If $X \sim f$:

$$\theta = \int g(x)f(x) dx = E[g(X)] \implies \hat{\theta} = \frac{1}{n} \sum_{i=1}^n g(X_i)$$

Two special cases cover most problems:

- **Integral on $[a, b]$:** $\int_a^b h(x) dx = (b - a) E[h(U)]$ with $U \sim \text{Uniform}(a, b)$.
- **Probability:** $P(X \in A) = E[\mathbf{1}_A(X)]$, which is the fraction of draws that land in A .

Method

1. Draw $X_1, \dots, X_n$ from f .
2. Compute $g(X_1), \dots, g(X_n)$.
3. Estimate $\hat{\theta}$ as the mean of these values.
4. Compute $\text{SE} = s_g/\sqrt{n}$, where s_g is the sample standard deviation of the $g(X_i)$.
5. Report the 95% confidence interval $\hat{\theta} \pm 1.96 \text{SE}$.

The estimator is unbiased. By the law of large numbers it is consistent, and by the central limit theorem the interval in step 5 is valid.

Example 1: an integral

Estimate $\int_0^1 e^{-x} dx$. The true value is $1 - e^{-1} = 0.6321$.

```
n <- 10000
x <- runif(n)                # random points on [0, 1]
g <- exp(-x)

est <- mean(g)
se <- sd(g) / sqrt(n)
```

```
c(estimate = est, se = se, lower = est - 1.96 * se, upper = est + 1.96 * se,
  truth = 1 - exp(-1))
```

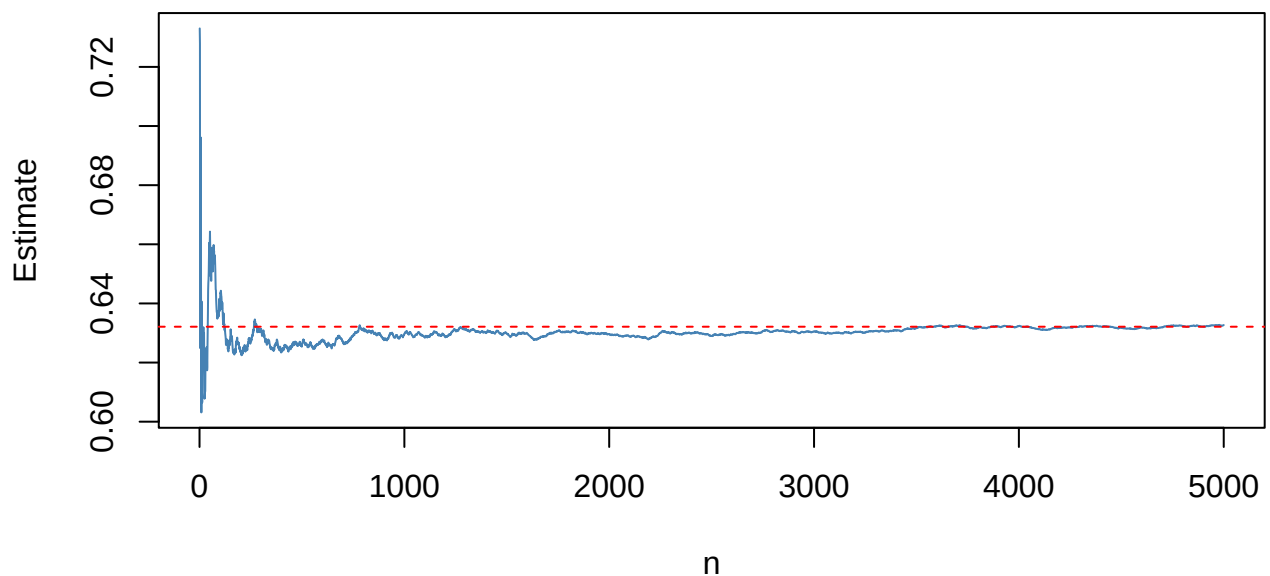
```
##      estimate      se      lower      upper      truth
## 0.633305690 0.001800576 0.629776561 0.636834820 0.632120559
```

The interval covers the true value. The next plot shows the estimate settling down as n grows.

```
n <- 5000
g <- exp(-runif(n))
running <- cumsum(g) / seq_len(n)

plot(running, type = "l", col = "steelblue", xlab = "n", ylab = "Estimate",
     main = "Monte Carlo estimate converging")
abline(h = 1 - exp(-1), col = "red", lty = 2)
```

Monte Carlo estimate converging



Example 2: a probability

Estimate $P(U_1 + U_2 > 1.5)$ for independent $U_1, U_2 \sim \text{Uniform}(0, 1)$. Elementary geometry gives the true value 0.125.

```
n <- 100000
u1 <- runif(n)
u2 <- runif(n)

hit <- (u1 + u2) > 1.5      # TRUE/FALSE for each draw
c(estimate = mean(hit), se = sd(hit) / sqrt(n), truth = 0.125)
```

```
##      estimate      se      truth
## 0.124780000 0.001045041 0.125000000
```

Importance Sampling

The problem

Suppose $\theta = P(Z > 4)$ with $Z \sim N(0, 1)$. The event is very rare, with probability about 3×10^{-5} . With 10,000 draws, plain Monte Carlo usually sees **no hits at all** and returns exactly 0.

For a small probability θ , the relative error of plain Monte Carlo is

$$\frac{\text{SE}}{\theta} \approx \frac{1}{\sqrt{n\theta}},$$

so you need $n \gg 1/\theta$ draws.

The idea

Sample from a different density p that visits the important region often. Then correct each draw with a **weight**:

$$\hat{\theta}_{IS} = \frac{1}{n} \sum_{i=1}^n g(Y_i) w(Y_i), \quad w(y) = \frac{f(y)}{p(y)}, \quad Y_i \sim p$$

This is still unbiased, because $E_p[g(Y) f(Y)/p(Y)] = \int g f dy = \theta$. The only requirement is that $p(y) > 0$ wherever $g(y)f(y) \neq 0$.

How to choose p

The variance is smallest when $p \propto |g| f$. You can't sample from that exactly, because it needs the answer you are looking for. So the rule of thumb is to **pick p shaped like $|g|f$** . For a tail probability, that means shifting the density toward the tail.

Method

1. Choose a proposal p that puts more mass on the important region, with tails at least as heavy as f 's.
2. Draw $Y_1, \dots, Y_n$ from p .
3. Compute the weights $w_i = f(Y_i)/p(Y_i)$.
4. Compute the values $g(Y_i) w_i$.
5. The estimate is their mean, and the SE is their standard deviation divided by $\sqrt{n}$.

Example 3: a rare event with importance sampling

For $P(Z > 4)$, use the proposal $N(4, 1)$, which is centred at the threshold.

```
n <- 10000
a <- 4

# Plain Monte Carlo
x <- rnorm(n)
g_naive <- as.numeric(x > a)

# Importance sampling
y <- rnorm(n, mean = a)          # draw from p = N(4, 1)
w <- dnorm(y) / dnorm(y, mean = a) # w = f / p
g_is <- (y > a) * w
```

```
data.frame(
  method = c("Plain MC", "Importance"),
  estimate = c(mean(g_naive), mean(g_is)),
  se = c(sd(g_naive) / sqrt(n), sd(g_is) / sqrt(n)),
  truth = 1 - pnorm(a)
)
```

```
##      method      estimate      se      truth
## 1 Plain MC 1.000000e-04 1.000000e-04 3.167124e-05
## 2 Importance 3.187168e-05 6.772927e-07 3.167124e-05
```

Plain Monte Carlo gives 0 with SE 0, which looks precise but is useless. Importance sampling lands close to the truth with a tiny standard error, using the same n .

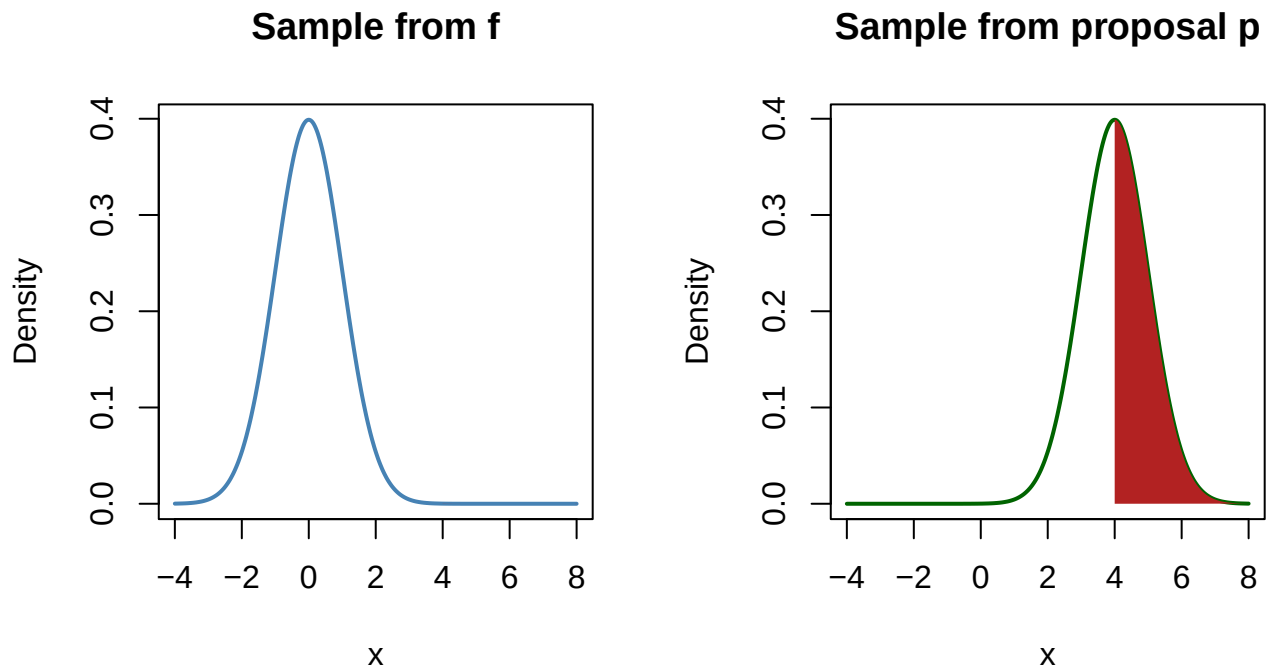


Figure 1: Left: $f = N(0,1)$ rarely reaches the shaded tail. Right: the proposal $N(4,1)$ reaches it half the time.

Check the weights

If a few weights are huge and the rest are tiny, the proposal is a bad match. A quick check is the **effective sample size**:

$$n_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}$$

If n_{eff} is much smaller than n , be suspicious of the estimate.

```
w_all <- w * (y > a)                                     # weights on the draws that matter
c(n = n, n_eff = sum(w_all)^2 / sum(w_all^2))
```

```
##      n      n_eff
## 10000.000 1813.094
```

Self-Normalized Importance Sampling

Sometimes f is known only up to a constant: $f(x) = \tilde{f}(x)/c$ with c unknown. This is typical for Bayesian posteriors. The constant cancels if you divide by the sum of the weights:

$$\hat{\theta}_{SNIS} = \frac{\sum_i g(Y_i) \tilde{w}_i}{\sum_i \tilde{w}_i}, \quad \tilde{w}_i = \frac{\tilde{f}(Y_i)}{p(Y_i)}$$

It is slightly biased for finite n , but consistent.

Example 4: unknown normalizing constant

Take the unnormalized target $\tilde{f}(x) = e^{-x^2/2}$ and estimate $E[X^2]$. The true value is 1, because the target is $N(0, 1)$. Use the proposal $N(0, 2^2)$.

```
n <- 50000
y <- rnorm(n, 0, 2)           # draws from p
w <- exp(-y^2 / 2) / dnorm(y, 0, 2) # unnormalized weights

sum(y^2 * w) / sum(w)        # SNIS estimate (truth = 1)
```

```
## [1] 1.000764
```

We never used the constant $\sqrt{2\pi}$.

Summary

	Monte Carlo	Importance sampling
Draw from	f	p (chosen by you)
Estimate	mean of $g(X)$	mean of $g(Y) f(Y)/p(Y)$
SE	$s/\sqrt{n}$	$s/\sqrt{n}$ (of weighted values)
Best for	simple or high-dimensional integrals	rare events, high variance
Best p		$\propto g f$

Tips

- Always report a standard error.
- The proposal's tails must be at least as heavy as the target's. Otherwise the weights can have infinite variance.
- Check the weights or n_{eff} .
- Use self-normalized weights when the density is known only up to a constant.
- For smooth, low-dimensional integrals, quadrature is faster. Monte Carlo wins as the dimension grows.

Exercises

1. Estimate $\int_0^1 x^2 dx$ by Monte Carlo and give a 95% interval. The true value is $1/3$.
2. Estimate π by simulation. Draw (U_1, U_2) uniform on the unit square and use $4 \times P(U_1^2 + U_2^2 < 1)$.
3. Estimate $P(Z > 3)$ both ways. What is the ratio of the variances?
4. In Example 3, change the proposal to $N(2, 1)$ and then to $N(6, 1)$. Compare the SE and n_{eff} .
5. Use a proposal with a lighter tail than the target (target t_3 , proposal $N(0, 1)$) to estimate $E[X^2]$. Look at the weights.